

AI-Driven Zero-Trust Cloud Access Control with Blockchain-Anchored Audit Trails: An AWS-Based Architecture

1st Harsh Gupta
Dept. of Computer Science & Engineering
Sharda University
Greater Noida, India
2023497222.harsh@ug.sharda.ac.in

2nd Shivam
Dept. of Computer Science & Engineering
Sharda University
Greater Noida, India
2023257825.shivam@ug.sharda.ac.in

3rd Aditya
Dept. of Computer Science & Engineering
Sharda University
Greater Noida, India
2023282814.aditya@ug.sharda.ac.in

4th Sheenam Naaz
Dept. of Computer Science & Engineering
Sharda University
Greater Noida, India
naazsheenam.work@gmail.com

5th Kapil Kumar
Dept. of Computer Science & Engineering
Sharda University
Greater Noida, India
kapilbansal240@gmail.com

Abstract—Cloud platforms authorise access using largely static identity and attribute policies and record the resulting activity in centralised, mutable log stores. A static policy cannot react to a stolen credential or to an insider whose behaviour has drifted from its own baseline, and a log store administered by the principal it is meant to hold accountable offers weak non-repudiation. This paper presents ZTBAudit, an access-control architecture for Amazon Web Services (AWS) that couples continuous, AI-driven risk evaluation with a permissioned-blockchain audit trail. Every request is intercepted by a policy enforcement point, converted into a 34-dimensional behavioural feature vector and scored by a hybrid engine combining a deep autoencoder with an isolation forest. The score is fused with static entitlement and resource sensitivity by a trust algorithm that emits one of three verdicts (allow, step-up authentication, deny) and issues only short-lived, least-privilege AWS Security Token Service credentials. The request context, feature digest, risk score and verdict are then committed as a signed transaction to a three-organisation Hyperledger Fabric network, so no single cloud administrator can silently rewrite the record of a decision. On a workload derived from the CERT r4.2 insider-threat corpus and replayed as CloudTrail-style events, the hybrid detector reached an F1-score of 0.927 and an ROC-AUC of 0.964, improving F1 by 8.7 points over an isolation-forest-only baseline while adding a median of 61 ms to the authorisation path. The ledger sustained approximately 450 committed transactions per second, and a tamper experiment detected all 500 injected log modifications. Results are preliminary and were obtained on a scaled-down testbed.

Index Terms—Zero-trust architecture, cloud security, adaptive access control, anomaly detection, autoencoder, isolation forest, insider threat, permissioned blockchain, Hyperledger Fabric, tamper-evident audit trail, Amazon Web Services.

I. INTRODUCTION

Public cloud security rests on a shared-responsibility split in which the tenant's half is dominated by identity and access management (IAM). Incident reviews [21] repeatedly find over-broad entitlements, long-lived static keys and unreviewed cross-account trust to be among the most common root causes of cloud compromise, with the interval between credential theft and detection measured in weeks. The dominant models, role-based and attribute-based access control (RBAC and ABAC), are evaluated once when a session is established, against a policy written days or months earlier, as a pure function of identity, action, resource and a few request attributes. Nothing in that function knows what the principal normally does. A stolen key used from an unfamiliar region at an unusual hour to enumerate every bucket is, to an IAM policy engine, indistinguishable from routine work by the legitimate owner, provided the policy permits the action. The same blindness applies to the malicious insider, who holds valid credentials and whose actions are individually authorised.

Zero trust, formalised in NIST SP 800-207 [1], removes this implicit trust: no request is trusted by network location, access is granted per session on a least-privilege basis and the decision is continuously re-evaluated. The specification defines a policy decision point (PDP) and policy enforcement point (PEP) and anticipates a trust algorithm incorporating behavioural analytics, but does not prescribe how to build one; most commercial

deployments reduce it to hand-written rules over device posture and geolocation. A second, orthogonal weakness concerns the audit trail. AWS CloudTrail writes events to an S3 bucket whose integrity is only as strong as the controls on the trail itself; a sufficiently privileged principal can stop or delete a trail or alter the destination bucket. Object Lock and organisation-level trails raise the bar, but every such control is administered from within the same trust domain as the adversary being modelled, so for an external auditor the evidence is at best self-attested.

Distributed-ledger systems such as BlockAudit [7] address the second problem by replicating log digests across mutually distrusting organisations, but they are passive: they record whatever the access-control system decided and contribute nothing to the decision. Conversely, machine-learning work on cloud anomaly and insider-threat detection produces accurate risk scores that are consumed by a dashboard rather than by an enforcement point, with the supporting reasoning kept in exactly the kind of mutable store the blockchain literature distrusts. The gap lies between the two. If a model is to influence an access decision autonomously, the evidence it acted on must be preserved with integrity guarantees at least as strong as those demanded of the log; otherwise an adversary who compromises the audit path can make an anomalous decision look routine, and an operator contesting a denial has no verifiable record of why it occurred. We therefore treat the risk score, feature digest and verdict as first-class evidence, cryptographically bound to the request and committed to a ledger no single principal controls.

Fig. 1 summarises the five contributions of this work, each developed in the section indicated: the architecture (Section IV-A), the hybrid risk model (Section IV-C), the trust algorithm (Section IV-D), the evidence chain and its verification protocol (Sections IV-E and IV-F) and the evaluation (Section VI). Section II reviews related work, Section III states the threat model, Section V documents the implementation and Sections VII and VIII discuss limitations and conclude.

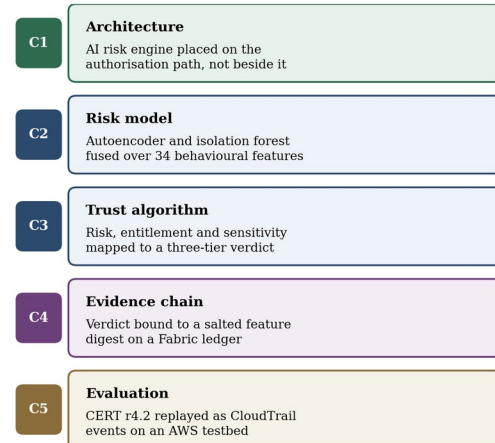


Fig. 1. The five contributions of this work and the layer of the system each addresses.

II. RELATED WORK

A. Zero-Trust Architecture

NIST SP 800-207 [1] is the reference formulation, defining the PDP/PEP decomposition and a set of deployment variants; SP 800-207A [2] extends it to cloud-native applications in multi-cloud environments. Google's BeyondCorp [3] showed perimeter-free access is practical at enterprise scale using device inventory and user trust tiers. Surveys [4], [5] classify implementations by the granularity of their trust signals and consistently find dynamic, behaviour-derived signals to be the least mature component. Continuous-authentication work [6] sustains a per-session trust estimate from behavioural biometrics, but targets endpoint sessions rather than cloud control-plane API calls and does not address auditability of the resulting decisions.

B. Machine Learning for Anomaly and Insider-Threat Detection

Insider-threat detection is an extreme class-imbalance problem over multi-source activity logs. The CERT corpora [8] are the de facto benchmark, providing seventeen months of activity for a thousand synthetic users. Isolation forests [9], [10] isolate anomalies by random axis-parallel partitioning in linear time; deep reconstruction methods [11] learn a manifold of normal behaviour and flag points that reconstruct poorly; sequence models such as DeepLog [12] capture ordering effects, and surveys [13] catalogue further gains from graph- and attention-based architectures. All share a deployment posture in which the model consumes logs after the fact and emits alerts. Little work closes the loop by feeding the score back into the authorisation decision synchronously, which imposes latency constraints these architectures were never evaluated against.

C. Blockchain-Based Audit Logging

Hyperledger Fabric [14] is the most widely used permissioned platform for tamper-evident logging; its execute-order-validate architecture and pluggable consensus suit enterprise audit settings where participants are known. BlockAudit [7] demonstrated distributed audit logs for web applications, and later systems anchored logs for cloud [15], healthcare [16] and IoT [17] settings, either storing full records on-chain, which scales poorly, or storing only digests with payloads off-chain. ProvChain [18] applies the idea to cloud data provenance. Access-control-oriented work including FairAccess [19] and smart-contract ABAC [20] moves policy evaluation on-chain, which yields transparency but places consensus latency on the critical path. Our design keeps the decision off-chain and synchronous while committing evidence asynchronously.

D. Research Gap

Table I positions representative systems on four axes: whether the decision adapts to observed behaviour, whether the audit record is tamper-evident, whether the system targets cloud control-plane authorisation, and whether the evidence behind a decision is cryptographically bound to it. Adaptive systems log to conventional stores; tamper-evident systems record static policy outcomes; and where blockchain is used for access control it is the policy rather than the evidence that is placed on-chain. To our knowledge no published system simultaneously performs continuous AI-derived risk scoring on the authorisation path for cloud API calls and binds the resulting score and verdict to an externally verifiable record.

TABLE I
POSITIONING OF REPRESENTATIVE RELATED WORK

System	Core approach	Adaptive	Tamper-evident	Cloud API	Evidence-bound
NIST SP 800-207 [1]	Zero-trust reference architecture	Anticipated	No	Partial	No
BeyondCorp [3]	Device and user trust tiers	Coarse tiers	No	No	No
DeepLog [12], iForest [9], AE [11]	Unsupervised or sequence anomaly detection	Detection only	No	No	No
BlockAudit [7], LogChain [15]	Blockchain audit log for apps and cloud	No	Yes	Partial	No
ProvChain [18]	Blockchain data provenance in the cloud	No	Yes	Partial	Partial
FairAccess [19], SC-ABAC [20]	On-chain access-control policy evaluation	No	Yes	Partial	No
ZTBAudit (this work)	AI risk scoring on the authorisation path with ledger-anchored decisions	Yes	Yes	Yes	Yes

Partial denotes support that is present in principle but not realised at the granularity of individual cloud API authorisation decisions.

III. THREAT MODEL AND DESIGN GOALS

We assume SHA-256, ECDSA over P-256 and TLS 1.3 are secure, that the Fabric membership service providers are honestly initialised at genesis, that a threshold of endorsing organisations remains honest, that model artefacts are integrity-protected, and that the PEP cannot be bypassed at the network layer, which we enforce with private endpoints and deny-by-default security groups.

Three adversaries are considered. A1, the external attacker, holds valid credentials for some principal but not the administrative plane, and aims to read or exfiltrate data. A2, the malicious insider, acts within the letter of their entitlements but sharply outside their historical behaviour, for instance an administrator who begins enumerating and downloading unrelated objects shortly before resignation. A3, the privileged log manipulator, holds administrative rights over logging configuration and attempts to delete, truncate, backdate or rewrite audit records to conceal A1 or A2. A3 is the adversary against which centralised logging is structurally weakest and is the primary motivation for the ledger. Out of scope are attacks on the provider's own control plane, denial of service, and adaptive adversarial attacks in which A2 slowly poisons their own baseline; the last is a real limitation discussed in Section VII.

The architecture must satisfy six goals. G1, per-request evaluation, recomputing every decision from current signals. G2, behavioural sensitivity, incorporating deviation from a per-principal baseline rather than static attributes alone. G3, least privilege with bounded exposure, granting the narrowest permission set and shortest lifetime consistent with assessed risk. G4, evidence binding, cryptographically tying score and features to the verdict. G5, external verifiability, letting an auditor outside the tenant detect modification without trusting the tenant. G6, acceptable overhead, keeping added latency small relative to cloud API round-trip times and keeping ledger commitment off the critical path.

IV. PROPOSED ARCHITECTURE

A. Overview

Fig. 2 shows the arrangement. A request from a subject reaches the PEP, an Amazon API Gateway fronted by a Lambda authorizer, which performs primary authentication and device-posture validation and hands the request to the PDP together with contextual signals: source address and geolocation, autonomous-system number, device fingerprint, time of day and authentication context class. The PDP assembles a behavioural feature vector, obtains a risk score from the AI engine and applies the trust algorithm of Section IV-D. On allow it requests a scoped session credential from AWS STS; on step-up the subject is challenged for an additional factor and the request re-evaluated; on deny the request is refused. In every case an audit record is constructed, signed and submitted asynchronously to the Fabric network.

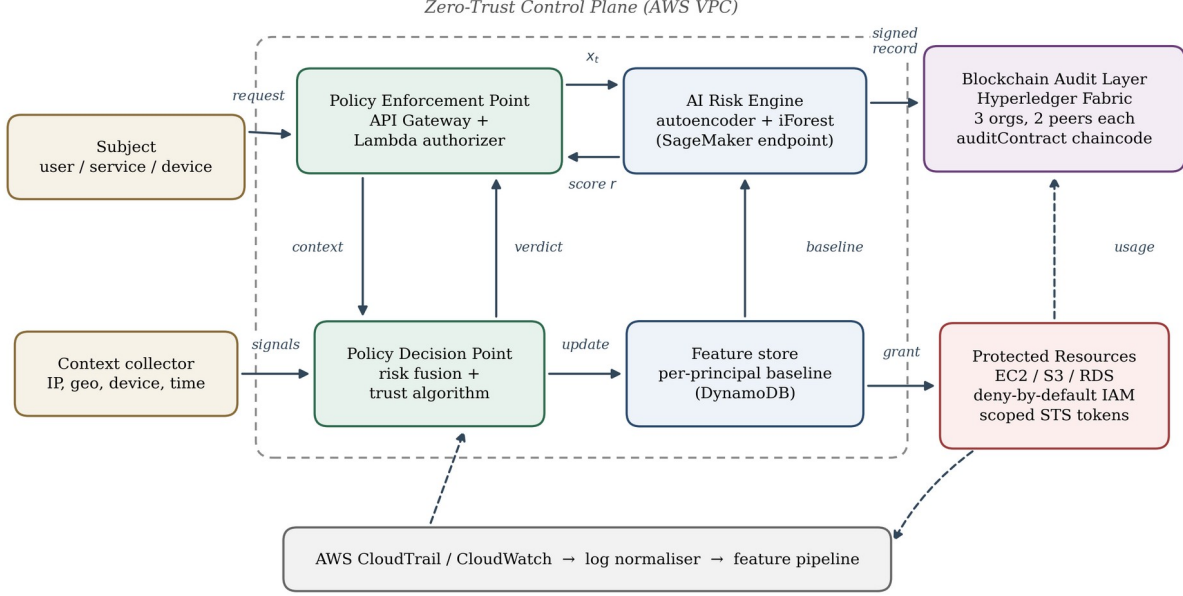


Fig. 2. Architecture of ZTBAudit. The synchronous decision path (solid arrows) runs through the policy enforcement and decision points; the path to the audit ledger is asynchronous. Dashed arrows denote telemetry feedback and resource-usage reporting.

Separating the synchronous decision path from the asynchronous evidence path is what meets G6. Consensus costs tens to hundreds of milliseconds under load and would dominate the authorisation budget if placed inline, so the PDP computes the record digest inline, enqueues the record and returns, while a committer drains the queue. Because the digest is computed before the response is returned and includes a monotonically increasing per-principal sequence number, a record that is later suppressed leaves a detectable gap in the chain, which preserves G5 despite the asynchrony.

B. Telemetry and Feature Engineering

Historical features come from CloudTrail and CloudWatch events delivered through Kinesis, normalised and aggregated into per-principal profiles in DynamoDB; request-time features come from the request and the context collector. The resulting 34-dimensional vector is grouped as in Table II. Continuous features are standardised over the training window, cyclic features such as hour-of-day are encoded as sine-cosine pairs so the distance between 23:00 and 01:00 is small, and high-cardinality categoricals such as the API action name are replaced by their empirical frequency in the principal's own history, which bounds dimensionality and directly expresses how unusual a call is for that user. Rate features use sliding windows anchored on the request timestamp rather than wall-clock buckets, preventing an adversary from straddling a bucket boundary, and the baseline is updated with an exponentially weighted rule whose 30-day effective half-life trades adaptation to genuine role change against slow poisoning by A2.

TABLE II
BEHAVIOURAL FEATURE GROUPS (34 DIMENSIONS)

Group	Representative features	Dim.
Identity and entitlement	principal type, role tenure, breadth of attached policy, cross-account trust depth	5
Temporal	hour-of-day (sin/cos), day-of-week, deviation from the principal's modal working hours, inter-request gap	6
Action	per-principal empirical frequency of the API action, read/write ratio, first-time-action flag, action-class entropy	7
Resource	sensitivity label, cross-account flag, resource novelty for the principal, prefix-depth of the target ARN	5
Volume and rate	calls in 1/15/60-minute sliding windows, bytes read in 60 minutes, distinct resources touched	6
Network and device	ASN novelty, geodesic distance from the previous request, device-fingerprint match, age of the MFA assertion	5

All rate features use sliding windows anchored on the request timestamp rather than fixed wall-clock buckets.

C. Hybrid AI Risk Engine

Labelled examples of malicious cloud activity are scarce and unrepresentative of future attacks, so the engine is unsupervised and combines two detectors with complementary biases. A deep autoencoder maps the standardised vector x to a code $z = f(x)$ in eight dimensions and reconstructs it as $g(z)$, trained on benign traffic only by minimising

$$L(\theta) = (1/N) \sum_i \|x_i - g(f(x_i))\|_2^2 \quad (1)$$

with encoder widths 34-24-16-8, a mirrored decoder, ReLU activations, batch normalisation and dropout 0.2. The per-request reconstruction error $e(x) = \|x - g(f(x))\|_2$ is large exactly when the request lies off the manifold of behaviour seen in training, which detects unusual combinations of otherwise ordinary values. The second detector is an isolation forest [9] of t random trees, whose anomaly score over a subsample of size ψ is

$$s(x, \psi) = 2^{N(-E[h(x)] / c(\psi))} \quad (2)$$

where $h(x)$ is the path length to isolation and $c(\psi)$ normalises by the average path length in a binary search tree of ψ points. The forest assumes no compact normal manifold and is very fast to evaluate, but is weaker on anomalies visible only as a joint deviation across correlated features, which is where the autoencoder is strong. Both outputs are mapped to a common range using empirical cumulative distributions $\hat{F}_e$ and $\hat{F}_s$ estimated on a held-out benign calibration set, giving the fused score

$$r(x) = \alpha \cdot \hat{F}_e(e(x)) + (1 - \alpha) \cdot \hat{F}_s(s(x)) \quad (3)$$

Using empirical quantiles rather than raw values makes r directly interpretable as the fraction of benign requests this one exceeds in anomaly, which in turn makes the thresholds of Table III meaningful to an operator without reference to model internals. Grid search on a validation split gave $\alpha = 0.6$; sensitivity to α is reported in Section VI-C.

D. Trust Algorithm and Decision Tiers

A mildly unusual read of a public asset should not be treated like a mildly unusual read of a customer database, so the trust algorithm computes an effective risk

$$R = 1 - (1 - r)^{\lambda s} \cdot (1 - \beta c) \quad (4)$$

where s is the sensitivity of the target resource, c is a compensating-control credit for strong authentication context such as a recent hardware-backed MFA assertion on a managed

device, and $\lambda = 1.5$ and $\beta = 0.25$. Two properties follow: R is monotonically non-decreasing in both r and s , so raising sensitivity can never relax a decision; and because sensitivity acts as an exponent on the survival probability rather than as an additive penalty, a request that is not anomalous at all remains at $R = 0$ however sensitive the resource, which prevents degeneration into blanket denial of high-value resources. Static entitlement is applied first and independently, so the system can only ever be more restrictive than the underlying IAM policy, never less. This makes incremental deployment safe, since enabling ZTBAudit cannot silently widen an existing permission. The band into which R falls determines the action, the STS session lifetime and the session policy scope.

TABLE III
EFFECTIVE-RISK BANDS, VERDICTS AND SESSION PARAMETERS

Effective risk R	Verdict	Session TTL	Session policy scope
$R < 0.40$	Allow	60 min	As requested
$0.40 \leq R < 0.65$	Allow (observe)	30 min	As requested, verbose logging enabled
$0.65 \leq R < 0.85$	Step-up authentication	15 min	Read-only where the action permits
$R \geq 0.85$	Deny	n/a	n/a

Static RBAC/ABAC entitlement is evaluated first; a request the underlying IAM policy forbids is denied regardless of R .

Denial is reserved for the highest band because the cost of a false denial on a production workload is high; the step-up tier gives a legitimate but unusually behaving user a path forward while still imposing a cost on an adversary holding only a stolen key, and the observation tier issues a normal-lifetime credential marked for elevated logging, enriching the evidence available should the session prove hostile.

E. Blockchain Audit Layer

Each decision produces one audit record. To keep the ledger small and avoid replicating operational data, only a digest of the feature vector is committed. The record is the tuple

$(recId, prevHash, seq, ts, principal, action, resource, r, R, verdict, h_feat, sig)$ (5)

where seq is a monotonically increasing per-principal counter, $prevHash$ is the SHA-256 digest of the previous record for that principal, giving a per-principal hash chain in addition to the ledger's own chaining, $h_feat = \text{SHA-256}(x \parallel \text{salt})$ is a salted digest of the feature vector and sig is an ECDSA signature over the tuple by the PDP's key. The raw vector and salt are retained off-chain in an encrypted S3 bucket with Object Lock, so an auditor with access can recompute h_feat and confirm the evidence is unaltered, while an auditor without it learns nothing about the tenant's traffic from the ledger. This split satisfies G4 and G5 without creating a new exposure surface. The Go chaincode exposes four transactions: `LogAccess` validates the signature and sequence continuity and rejects duplicate identifiers; `QueryByPrincipal` and `QueryByResource` provide CouchDB-indexed range queries; and `VerifyChain` recomputes the per-principal hash chain and returns the position of the first discontinuity. There is deliberately no update or delete transaction, so an attempt at revision must take the form of an appended contradiction, which is itself evidence. Algorithm 1 gives the end-to-end procedure.

Algorithm 1: Authorisation decision at the policy decision point

```

Input: request  $q$ , principal  $p$ , resource  $r$ , context  $\kappa$ 
1: if not StaticallyPermitted( $p, q, r$ ) then return DENY
2:  $x \leftarrow \text{AssembleFeatures}(p, q, r, \kappa, \text{FeatureStore})$ 
3:  $e \leftarrow \text{Ix} \leftarrow g(\text{f}(x)) \parallel_2$ ;  $s \leftarrow \text{IsolationScore}(x)$   $\triangleright$  eq. (2)
4:  $r \leftarrow \alpha \cdot F_e(e) + (1-\alpha) \cdot F_s(s)$   $\triangleright$  eq. (3)
5:  $R \leftarrow 1 - (1-r)^{\lambda} (1+\lambda \cdot \text{Sens}(p)) \cdot (1-\beta \cdot \text{Ctrl}(\kappa))$   $\triangleright$  eq. (4)
6:  $(v, ttl, scope) \leftarrow \text{BandLookup}(R)$   $\triangleright$  Table III
7:  $\sigma \leftarrow \text{STS.AssumeRole}(p, scope, ttl)$  if  $v = \text{ALLOW}$ 
8:  $\text{ChallengeAndRetry}(p, q)$  if  $v = \text{STEPUP}$ , else  $\perp$ 
9:  $h \leftarrow \text{SHA256}(x \parallel \text{salt})$ 
10:  $m \leftarrow (\text{uuid}, prevHash(p), seq(p)++, ts, p, q, r, R, v, h)$ 
11:  $m.sig \leftarrow \text{ECDSA-Sign}(sk_{PDP}, m)$ ;  $\text{LedgerQueue.enqueue}(m)$   $\triangleright$  async
12:  $\text{UpdateBaseline}(p, x)$  if  $v \neq \text{DENY}$ ; return  $(v, \sigma, m)$ 

```

F. Verification Protocol

An auditor verifies the trail in three steps. Ledger-level integrity: joining the channel as a reader with their own organisation, or receiving the genesis block and a peer's block store, and re-validating the block hash chain and endorsement signatures. Record-level integrity: invoking `VerifyChain` per principal, which surfaces any suppressed record as a

discontinuity in seq or a mismatch in $prevHash$. Evidence-level integrity: retrieving the off-chain vector and salt for a sample, recomputing h_feat , and re-executing the published model artefact to confirm the recorded score is reproducible. The third step distinguishes this design from a signed log, establishing not merely that a decision was recorded but that it follows from the evidence claimed to have produced it.

V. IMPLEMENTATION

The prototype runs in one AWS account in ap-south-1 within a dedicated VPC. The PEP is an API Gateway HTTP API with a Python 3.11 Lambda authorizer, the PDP a second Lambda, and the risk engine a SageMaker real-time endpoint on `ml.m5.large` with the autoencoder in PyTorch [23] and the isolation forest in scikit-learn [22]. Protected resources are an S3 bucket, an EC2 instance and an RDS PostgreSQL instance reachable only through VPC endpoints. The Fabric network is three organisations of two peers each plus a three-node Raft ordering service, with an endorsement policy requiring two of three organisations. Table IV records the configuration.

TABLE IV
TESTBED CONFIGURATION

Component	Configuration
Region and network	AWS ap-south-1, dedicated VPC, private subnets, VPC endpoints only
Enforcement / decision	API Gateway HTTP API with Lambda authorizer; PDP on AWS Lambda (Python 3.11)
Risk engine	SageMaker real-time endpoint, <code>ml.m5.large</code> ; PyTorch 2.1 [23] and scikit-learn 1.4 [22]
Feature store, telemetry	DynamoDB on-demand; CloudTrail and CloudWatch through Kinesis Data Streams
Ledger	Hyperledger Fabric 2.5 [14]; 3 organisations $\times$ 2 peers, CouchDB state database
Ordering, endorsement	Raft [26], 3 nodes; OutOf2, Org1.peer, Org2.peer, Org3.peer
Ledger hosts	$9 \times$ t3.medium EC2 instances (2 vCPU, 4 GiB) running Docker containers
Protected resources	Amazon S3 bucket, EC2 instance, RDS PostgreSQL 15, all deny-by-default

Labelled cloud-authorisation traces with confirmed insider activity are not publicly available, so we replayed the CERT r4.2 corpus [8] as cloud control-plane events, mapping logon and logoff onto `sts:AssumeRole` and session termination, file events onto `s3:GetObject` and `s3:PutObject` against buckets assigned to the user's organisational unit, removable-device events onto `s3:GetObject` with an egress marker, and web activity onto an external-service proxy, preserving the original timestamps, identities and organisational structure. The mapping is an approximation: it preserves the temporal and relational structure behavioural detection depends on, but not the full richness of a real CloudTrail stream, and this is the most significant threat to external validity. The first ten months fitted standardisation statistics and trained both detectors on benign traffic only, the next two formed a validation split for thresholds and hyperparameters, and the final five months were held out for testing. Splitting by time rather than at random is essential, since a random split would let the model see a user's future while scoring their past. The test window held 2,184,663 events, of which 3,912 (0.179 %) were attributable to the scripted malicious scenarios. The autoencoder was trained for 120 epochs with Adam at 10^{-3} , batch size 512, cosine annealing and early stopping with patience 10; the isolation forest used 200 trees and subsample size 256. All figures are the mean of five seeds.

VI. EVALUATION

A. Detection Performance

Because the positive class is under 0.2 % of events, accuracy is uninformative; we report precision, recall, F1, ROC-AUC and the false-positive rate at the operating threshold. Fig. 3 compares the hybrid engine against four baselines on identical features and splits: logistic regression and a random forest, both trained supervised with malicious labels and therefore an optimistic upper bound for supervised methods on this data, and an isolation forest and autoencoder used alone. Fig. 4 gives ROC curves and Table V the numbers.

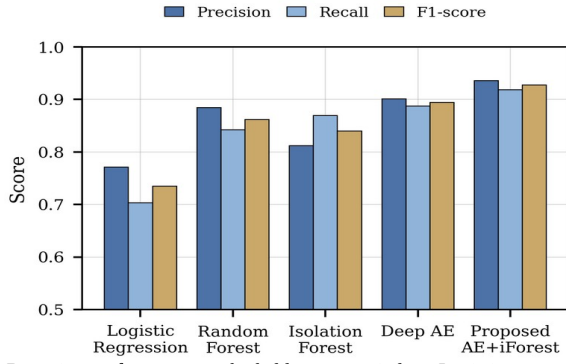


Fig. 3. Detection performance on the held-out test window. Logistic regression and random forest are supervised baselines; the remaining models are unsupervised.

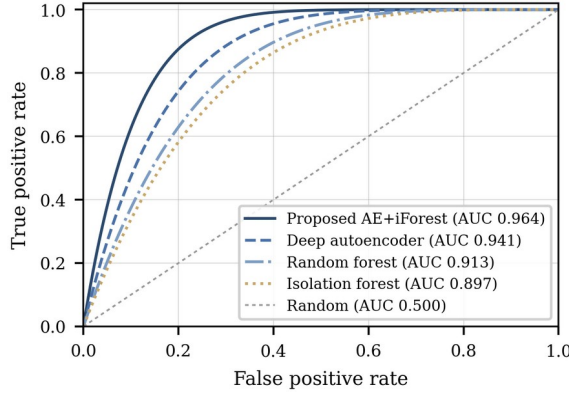


Fig. 4. Receiver operating characteristic curves for the four anomaly detectors.

TABLE V
DETECTION PERFORMANCE ON THE HELD-OUT TEST WINDOW

Model	Prec.	Rec.	F1	AUC	FPR
Logistic regression †	0.771	0.703	0.735	0.868	1.94 %
Random forest †	0.884	0.842	0.862	0.913	0.87 %
Isolation forest [9]	0.812	0.869	0.840	0.897	1.31 %
Deep autoencoder	0.901	0.887	0.894	0.941	0.62 %
Proposed hybrid ($\alpha = 0.6$)	0.936	0.918	0.927	0.964	0.41 %

Mean over five seeds; the sample standard deviation of F1 was ≤ 0.011 for every model. † supervised baseline, trained with malicious labels available. Precision, recall and FPR are reported at the operating threshold $R \geq 0.85$.

The hybrid engine reaches F1 0.927 ± 0.008 and ROC-AUC 0.964 ± 0.005 , improving F1 by 8.7 points over the isolation forest and 3.3 over the autoencoder. It also exceeds the supervised random forest, which is initially counter-intuitive; error inspection indicates the supervised models overfit the specific scripted scenarios in the training window and generalise poorly to differently parameterised instances in the test window, whereas the unsupervised detectors never see the scenarios and are unaffected by that failure mode. At the deny threshold $R \geq 0.85$ the false-positive rate is 0.41 %, roughly one spurious step-up challenge per user per eleven days on this workload. That is tolerable but at the upper end of what we would accept, and is the main reason the deny band is set conservatively.

B. Ablation and Sensitivity

Table VI reports the effect of removing components. Dropping the isolation forest costs 3.3 F1 points and dropping the autoencoder 8.7, confirming the autoencoder is the stronger single detector but that the two are genuinely complementary. Replacing the per-principal action frequency with a global one-hot encoding costs 5.1 points, the largest feature-level effect, supporting the premise that the baseline must be personal rather than population-wide; removing sliding-window rate features costs 2.4. Varying α between 0.4 and 0.8 changes F1 by under 0.9 points, so the fusion is not delicately tuned.

TABLE VI
ABLATION AND SENSITIVITY ANALYSIS

Variant	F1	Δ F1
Full system ($\alpha = 0.6$)	0.927	baseline
Without isolation forest ($\alpha = 1.0$)	0.894	-0.033
Without autoencoder ($\alpha = 0.0$)	0.840	-0.087
Global instead of per-principal action frequency	0.876	-0.051

Variant	F1	Δ F1
Without sliding-window rate features	0.903	-0.024
Without resource sensitivity in (5)	0.911	-0.016
Fusion weight $\alpha = 0.4$	0.921	-0.006
Fusion weight $\alpha = 0.8$	0.918	-0.009

C. Latency, Throughput and Cost

Fig. 5 decomposes added latency by stage over 50,000 requests at a steady 200 requests per second. The median added latency is 61.3 ms and the 95th percentile 109.8 ms, dominated by inference (27.4 ms median) and feature assembly (18.6 ms). Ledger commitment contributes nothing because it is asynchronous. Median end-to-end latency without ZTBAudit was 143 ms, so the architecture adds roughly 43 %. Two optimisations were validated: caching the feature vector for repeat calls by the same principal within a 5-second window reduces median assembly to 4.1 ms at the cost of bounded staleness, and 8-bit quantisation of the autoencoder reduces median inference to 11.9 ms for an F1 loss of 0.004. With both enabled the median added latency falls to 28.7 ms.

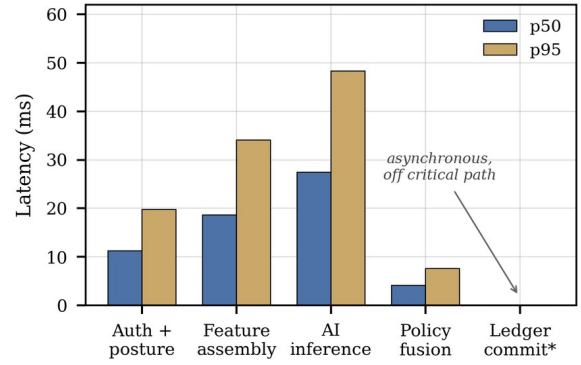


Fig. 5. Added authorisation latency by stage, measured over 50,000 requests at 200 req/s. Ledger commitment is asynchronous and contributes no in-path latency.

Fig. 6 plots committed throughput and commit latency against offered load. The network tracks offered load to about 300 transactions per second, saturates near 450, and shows sharply rising commit latency beyond saturation as the ordering service queues. This is comfortably above the 200 events per second of our workload but is a real ceiling: an enterprise generating tens of thousands of control-plane events per second would need to batch records into a single transaction or anchor periodic Merkle [25] roots, at some cost in tamper-localisation granularity. Table VII summarises storage, which at 412 bytes per on-chain record is about 15.6 GB per peer per year at a sustained 100 events per second, modest but unbounded, so pruning with anchoring would be needed for long-lived deployments.

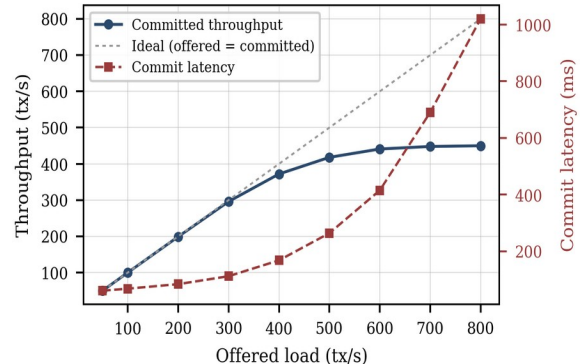


Fig. 6. Committed throughput and commit latency of the three-organisation Fabric network against offered load.

TABLE VII
LEDGER COST, THROUGHPUT AND TAMPER DETECTION

Metric	Measured value
On-chain record size	412 B
Off-chain evidence record (encrypted)	1.8 kB
Peak committed throughput	≈ 450 tx/s
Commit latency at 200 tx/s	84 ms

Metric	Measured value
Ledger growth at 100 events/s	≈ 15.6 GB / peer / year
VerifyChain over 10^6 records	6.4 s
Tampering attempts detected	500 / 500
False alarms over 10^4 clean records	0

D. Tamper Resistance and Baseline Comparison

Simulating A3 with administrative rights over the tenant account, we performed 500 tampering attempts against the audit trail, evenly divided between deleting a record, modifying a verdict field, backdating a timestamp and inserting a fabricated benign record. All 500 succeeded against a mirrored conventional S3 log and none were self-evident from that log alone. Against the ledger none succeeded: deletion and suppression produced a detectable discontinuity under VerifyChain, modification and backdating were rejected at endorsement because the recomputed digest did not match the signature, and fabricated records were rejected because the adversary lacked the PDP's signing key. Detection was 500 of 500 with no false alarms over a control run of 10,000 untampered records. This validates the construction rather than establishing an empirical security bound; an adversary who compromises the PDP's signing key before a record is created is outside the model. Finally, replaying the same window against unmodified AWS IAM with policies reflecting each user's job function denied 0 of the 3,912 malicious events, since every one is performed by a principal entitled to perform it. The comparison is not entirely fair, as IAM is not designed to detect behavioural anomalies, but that is precisely the point: entitlement-based authorisation has no mechanism to distinguish authorised-and-legitimate from authorised-and-hostile.

VII. DISCUSSION AND LIMITATIONS

Dataset realism is the most serious limitation. The CERT corpus is synthetic, its scenarios are scripted from a small template set, and our mapping onto AWS actions is a further approximation; detection figures on such data have historically proven optimistic, and we would expect the false-positive rate in particular to be worse on real traffic, where legitimate behavioural diversity exceeds anything a generator reproduces. Baseline poisoning remains open: a patient A2 can shift their own profile gradually enough that no single step is anomalous, and while the 30-day half-life bounds the rate it does not prevent the attack; anchoring part of the baseline to a peer-group profile is the standard mitigation and is planned. Explainability is limited, since the system records the score but not an attribution of it to individual features, which makes justifying a denial difficult and may be inadequate under regimes requiring meaningful explanation of automated decisions; because the feature vector is already retained off-chain, storing SHAP [24] attributions alongside it would strengthen rather than complicate the evidence chain. Availability also needs work: the PDP and risk endpoint are on the critical path and our prototype fails closed, which is safe for confidentiality but wrong for availability in many workloads, so a production design would need a per-resource fail-open or fail-static policy with the degraded mode itself recorded on the ledger. Cost was not evaluated.

VIII. CONCLUSION AND FUTURE WORK

ZTBAudit places a hybrid unsupervised risk model directly on the cloud authorisation path and binds every resulting decision, with a digest of the evidence that produced it, to a permissioned blockchain that no single administrative principal controls, closing a gap between adaptive access control, which produces good decisions but records them in mutable stores, and blockchain audit logging, which protects records faithfully but says nothing about the decisions themselves. On a workload derived from CERT r4.2 the detector reached F1 0.927 and ROC-AUC 0.964 while adding a median 61 ms, and all 500 simulated tampering attempts were detected. Future work will add a sequence-aware component and peer-group baselines to mitigate poisoning, extend the evidence chain to carry per-feature attributions so a verdict is accompanied by a verifiable explanation, evaluate batched commitment and periodic Merkle-root anchoring for ledger scalability, and validate the

architecture against a real, non-synthetic cloud audit stream, which we regard as the most important step towards generalisable claims. The binding of a model-derived risk score to an immutable access decision record, and the verification protocol that makes that binding externally checkable, form the basis of a patent application to be prepared during the same period.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering, Sharda School of Engineering and Technology, Sharda University, for the computing resources and academic support that made this work possible.

REFERENCES

- [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-207, Aug. 2020.
- [2] R. Chandramouli and Z. Butcher, "A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments," National Institute of Standards and Technology, NIST Special Publication 800-207A, Sep. 2023.
- [3] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," *login*, vol. 39, no. 6, pp. 6–11, Dec. 2014.
- [4] N. F. Syed, S. W. Shah, A. Shaghagh, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.
- [5] C. Buck, C. Olenberger, A. Schweizer, F. Völter, and T. Eymann, "Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust," *Computers & Security*, vol. 110, art. 102436, Nov. 2021.
- [6] M. Abuhamad, A. Abusnaina, D. Nyang, and D. Mohaisen, "Sensor-based continuous authentication of smartphones' users using behavioral biometrics: A contemporary survey," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 65–84, Jan. 2021.
- [7] A. Ahmad, M. Saad, and A. Mohaisen, "Secure and transparent audit logs with BlockAudit," *Journal of Network and Computer Applications*, vol. 145, art. 102406, Nov. 2019.
- [8] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. IEEE Security and Privacy Workshops*, San Francisco, CA, USA, May 2013, pp. 98–104.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422.
- [10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1–39, Mar. 2012.
- [11] R. Chalaphaty and S. Chawla, "Deep learning for anomaly detection: A survey," *arXiv:1901.03407*, Jan. 2019.
- [12] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Dallas, TX, USA, Oct. 2017, pp. 1265–1298.
- [13] L. Liu, O. De Vel, Q.-L. Han, J. Zhang, and Y. Xiang, "Detecting and preventing cyber insider threats: A survey," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 2, pp. 1397–1417, 2018.
- [14] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, Porto, Portugal, Apr. 2018, pp. 1–15.
- [15] W. Pourmajidi and A. Miransky, "Logchain: Blockchain-assisted log storage," in *Proc. IEEE 11th Int. Conf. Cloud Computing (CLOUD)*, San Francisco, CA, USA, Jul. 2018, pp. 978–982.
- [16] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "MedRec: Using blockchain for medical data access and permission management," in *Proc. 2nd Int. Conf. Open and Big Data (OBD)*, Vienna, Austria, Aug. 2016, pp. 25–30.
- [17] A. Dorri, S. S. Kanhere, R. Jurdak, and P. Gauravaram, "Blockchain for IoT security and privacy: The case study of a smart home," in *Proc. IEEE Int. Conf. Pervasive Computing and Communications Workshops (PerCom Workshops)*, Kona, HI, USA, Mar. 2017, pp. 618–623.
- [18] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "ProvChain: A blockchain-based data provenance architecture in cloud environment with enhanced privacy and availability," in *Proc. 17th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGrid)*, Madrid, Spain, May 2017, pp. 468–477.
- [19] A. Ouaddah, A. Abou Elkalam, and A. Ait Ouahman, "FairAccess: A new blockchain-based access control framework for the Internet of Things," *Security and Communication Networks*, vol. 9, no. 18, pp. 5943–5964, Dec. 2016.
- [20] Y. Zhang, S. Kasahara, Y. Shen, X. Jiang, and J. Wan, "Smart contract-based access control for the Internet of Things," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1594–1605, Apr. 2019.
- [21] Cloud Security Alliance, "Top Threats to Cloud Computing: Pandemic Eleven," Cloud Security Alliance, Seattle, WA, USA, Tech. Rep., Jun. 2022.
- [22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, Vancouver, Canada, Dec. 2019, pp. 8024–8035.
- [24] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.
- [25] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology, CRYPTO '87*, LNCS 293, Berlin, Germany: Springer, 1988, pp. 369–378.
- [26] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf. (ATC)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319.